



PROJECT BRIEF

Cahier des Charges

Plateforme de gestion de tableaux et collaboration

Version : 1.0 • Date : Février 2026

Sommaire

1. Résumé exécutif

2. Contexte, vision et objectifs

2.1 Contexte

2.2 Vision produit

2.3 Objectifs fonctionnels

2.4 Objectifs techniques

3. Périmètre fonctionnel (actuel + cible)

3.1 Déjà en place

3.2 Cible proche

3.3 Évolutions prévues

4. Architecture technique

4.1 Frontend

4.2 Backend

4.3 Infrastructure

5. Benchmark technologique (choix justifiés)

5.1 Frontend

5.2 Backend

5.3 API

5.4 Base de données

5.5 Infra

6. Environnements

6.1 Dev

6.2 Prod

7. CI/CD & Qualité

8. Organisation du projet (Git/GitHub)

9. Plan de livraison (macro)

10. Risques & mitigations

11. Livrables

Document évolutif, mis à jour au fil des itérations.

1. Résumé exécutif

EpiTrello est une application de gestion de projets inspirée de Trello, centrée sur des tableaux kanban, des workspaces et la collaboration. Le produit vise une expérience fluide et moderne, avec une base technique scalable et une organisation de projet rigoureuse (CI/CD, branches par feature, environnements dev/prod).

L'objectif est de livrer une première version solide (auth, tableaux, workspaces, paramètres) tout en préparant les évolutions stratégiques : temps réel, templates de tableaux, automatisations et notifications.

Le projet s'appuie sur une stack web moderne (Next.js, NestJS GraphQL, Prisma, PostgreSQL) et une infrastructure reproductible via Docker et Terraform. Les environnements dev/prod sont séparés, et la qualité est suivie via tests automatisés et rapports de couverture backend.

L'organisation du travail se fait par features (branches dédiées, issues et PR) afin de maintenir une cadence claire et une traçabilité des changements.

2. Contexte, vision et objectifs

2.1 Contexte

Le projet EpiTrello vise à proposer un outil de gestion visuelle du travail, inspiré des meilleures pratiques Kanban, tout en répondant aux contraintes académiques et professionnelles d'un projet de fin d'étude. L'enjeu n'est pas seulement fonctionnel : il s'agit aussi de démontrer une capacité à concevoir une architecture complète (front, back, infra), à industrialiser le développement et à livrer itérativement.

Le produit doit être utilisable dès les premières itérations, avec une base solide, testable et déployable, pour permettre une montée en charge progressive sans refonte majeure.

2.2 Vision produit

Offrir une alternative claire, rapide et cohérente à Trello : une interface lisible, des parcours simples, et une collaboration fluide. L'expérience utilisateur doit rester prioritaire, avec un design systémique et des composants réutilisables.

La vision est d'aboutir à une plateforme qui facilite la gestion d'équipes et de projets, avec un socle technique robuste et des fonctionnalités avancées prêtes pour le temps réel et l'automatisation.

2.3 Objectifs fonctionnels

- Créer et organiser des tableaux, listes et cartes de manière intuitive.
- Structurer la collaboration par workspaces (membres, rôles, permissions).
- Offrir une authentification moderne (email/mot de passe, OAuth).
- Proposer des paramètres utilisateur complets (profil, sécurité, accessibilité).
- Préparer le terrain pour l'activité et les notifications.

2.4 Objectifs techniques

- Mettre en place un front moderne (Next.js, design system, composants standardisés).
- Exposer une API GraphQL robuste, typée et testable.
- Garantir une infrastructure reproductible (Docker, Terraform, GCP).
- Industrialiser la qualité via CI/CD et couverture de code backend.
- Séparer clairement dev/prod pour sécuriser les déploiements.

3. Périmètre fonctionnel (actuel + cible)

3.1 Déjà en place

- Frontend Next.js (pages landing, login/signup, account settings, boards).
- Sidebar boards avec favoris/récents/workspaces.
- Pages de paramètres utilisateur (profil, activité, paramètres, accessibilité).
- Navbar commune + navigation unifiée.
- Base GraphQL et génération de types côté frontend (codegen).
- Stack dev complète avec Postgres, MinIO (S3), pgAdmin, Prisma Studio et docs API.

3.2 Cible proche

- Auth OAuth (GitHub/Google côté backend, Discord côté UI) + email/mdp.
- Gestion complète des workspaces (membres, rôles, paramètres, danger zone).
- Gestion des boards (création, visibilité, templates).
- Fichiers/avatars via stockage objet (MinIO en dev).

3.3 Évolutions prévues

- Temps réel via WebSocket (cartes, listes, commentaires).
- Templates de boards.
- Notifications et activité.
- Filtrage avancé et recherche.

4. Architecture technique

4.1 Frontend

- Next.js 14 + React 18.
- Tailwind CSS + shadcn/ui.
- Apollo Client pour GraphQL.

4.2 Backend

- NestJS + Apollo GraphQL (API principale).
- Auth JWT + OAuth (Google, GitHub).
- Prisma ORM + PostgreSQL.
- Stockage objet S3-compatible (MinIO en dev).

4.3 Infrastructure

- Docker Compose (dev/prod) + services annexes (pgAdmin, Prisma Studio, docs).
- Terraform (GCP) : Cloud Run, Cloud SQL, Secret Manager, Artifact Registry.
- Provider cloud : Google Cloud Platform (GCP).

● Schéma d'architecture (vue simplifiée)

Frontend

Next.js / React
UI • Routing • Auth UI

API GraphQL

Apollo Server
Resolvers • Auth •
Business logic

Base de données

PostgreSQL + Prisma
Workspaces • Boards •
Users

Infrastructure & Exécution

Docker Compose (dev/prod) • Terraform • GCP (Cloud Run, Cloud SQL, Secret Manager)
Services dev : MinIO (S3), pgAdmin, Prisma Studio, SpectaQL.

Flux principal : Frontend → API GraphQL → Base de données.

5. Benchmark technologique (choix justifiés)

5.1 Frontend

Next.js vs Vite+React: Next.js apporte SSR/SSG, routing intégré, et un déploiement simple. Vite est plus léger mais nécessite plus d'assemblage.

• Comparatif Frontend

Option retenue	Alternative	Synthèse
 Next.js	 Vite + React	SSR/SSG + routing intégré vs setup plus libre.

5.2 Backend

NestJS + Apollo vs Express + Apollo: NestJS apporte une structure claire (modules, DI, tests) tout en conservant Apollo pour GraphQL. Express reste plus léger mais demande plus de conventions internes.

• Comparatif Backend

Option retenue	Alternative	Synthèse
 NestJS + Apollo	 Express + Apollo	Structure claire + DI vs approche plus légère.

5.3 API

GraphQL vs REST: GraphQL réduit le sur-fetching, permet de composer les vues rapidement et convient bien aux interfaces complexes (boards, cartes, membres).

• Comparatif API

Option retenue	Alternative	Synthèse
 GraphQL	 REST	Moins d'over-fetching, vues composables.

5.4 Base de données

PostgreSQL vs MySQL/MongoDB: PostgreSQL est robuste pour les relations (workspaces, boards, permissions) et bien supporté par Prisma.

● Comparatif Base de données

Option retenue	Alternative	Synthèse
 PostgreSQL	 MySQL /  MongoDB	Relations robustes et compatibilité Prisma.

5.5 Infra

GCP + Terraform vs alternatives (AWS/Azure): choix motivé par la capacité à automatiser l'infra via Terraform et la cohérence avec les environnements de déploiement visés.

● Comparatif Infra

Option retenue	Alternative	Synthèse
 GCP +  Terraform	 AWS /  Azure	Déploiement reproductible, infra as code.

6. Environnements

6.1 Dev (local / Docker)

Objectif

Fournir un environnement complet et reproductible pour le développement quotidien, avec hot reload front/back.

Services inclus

- PostgreSQL (DB)
- Backend GraphQL (NestJS)
- Frontend Next.js
- MinIO (S3) + console
- pgAdmin + Prisma Studio
- Docs API (SpectaQL)
- Coverage backend (HTML)

Ports clés

3000 (front), 4000 (API), 5432 (DB), 5050 (pgAdmin), 5555 (Prisma), 9000/9001 (MinIO), 4400 (Docs), 7331 (Coverage).

6.2 Prod (déploiement)

Objectif

Exposer une version stable, sécurisée et scalable, alignée avec l'infrastructure GCP et les contraintes de production.

Services & Infra

- Frontend + Backend (containers)
- PostgreSQL (Cloud SQL en infra cible)
- Secrets via Secret Manager
- Images via Artifact Registry

Variables & sécurité

.env.prod (JWT, CORS, OAuth, DB). Séparation stricte des secrets prod et dev.

7. CI/CD & Qualité

État actuel (dans le repo)

- Tests backend avec Jest (``test``, ``test:cov``, ``test:cov:watch``).
- Rapport de couverture HTML exposé via container ``backend-coverage`` (port 7331).
- Badge Codecov affiché dans le README (indique un suivi de couverture).
- Lint frontend via ``next lint``.
- Génération de types GraphQL côté frontend (codegen).

CI/CD cible (pipeline attendu)

- Lint + tests automatiques à chaque PR.
- Publication des rapports de couverture (Codecov).
- Build d'images Docker (frontend/backend).
- Push sur Artifact Registry.
- Déploiement Cloud Run (GCP) via Terraform.

Pipeline cible (vue simplifiée)

Lint & Tests

Coverage

Build Images

Push Registry

Deploy GCP

8. Organisation du projet (Git/GitHub)

- Branching par **feature**.
- `main` = branche de production.
- PR obligatoires, review avant merge.
- Issues et PR gérées via **GitHub CLI** (`gh`), avec assignation et labels.
- Les PR indiquent les issues clôturées (`Closes #id`).

Règles de gestion

- Une issue = une feature ou un bloc fonctionnel clair.
- Une branche dédiée par issue (ex: `feature/nom`).
- Les commits suivent une granularité lisible (UI, logique, refactor).
- PR en anglais avec description, captures si besoin.
- Reviewer assigné avant merge.

Template de Pull Request (exigé)

Chaque PR doit fournir un contexte clair, l'impact sur le produit, et relier les issues traitées.

```
## Summary
- What was implemented?
- Why this change is needed?

## Changes
- [ ] UI
- [ ] Logic
- [ ] Refactor
- [ ] Tests

## Screenshots (if UI)
- Before / After

## Linked issues
- Closes #ID

## Notes
- Potential follow-ups or risks
```

Labels & traçabilité

- Labels principaux : **frontend**, **features**, **backend**.
- Clôture automatique via `Closes #id` dans la PR.
- Assignation de l'auteur sur l'issue et la PR.

Flux de travail

Issue

Branch
feature

Commits

PR

Review

Merge main

9. Plan de livraison (macro)

1. Stabilisation auth + pages publiques.
2. Boards + workspaces (CRUD, permissions).
3. Temps réel et activité.
4. Templates et automatisations.
5. Optimisations + monitoring.

Vue par phases

Phase 1
Auth &
Landing

Phase 2
Boards/Workspaces

Phase 3
Temps réel

Phase 4
Templates

Phase 5
Perf &
Observabilité

10. Risques & mitigations

- **OAuth instable** : config stricte des redirect URI, tests multi-env.
- **Temps réel** : introduire progressivement via WebSocket + feature flags.
- **Complexité UI** : composants UI standardisés (shadcn/ui).
- **Dépendances infra** : mocks locaux (Docker/MinIO), plan de rollback.
- **Dette technique** : conventions strictes (PR + review), refactoring itératif.

11. Livrables

- Frontend complet (Next.js).
- Backend GraphQL (NestJS) + DB (PostgreSQL).
- Infra Terraform GCP (Cloud Run, Cloud SQL, Secret Manager).
- Fichiers de déploiement Docker Compose (dev/prod).
- Documentation technique (README, docs API, Postman OAuth).
- Tests backend + couverture.

Document évolutif, mis à jour au fil des features et livraisons.